

LayoverNow Project Proposal

Platform: Chrome Extension (Manifest V3, JavaScript)

Repository: GitHub, with instructor access granted

Development environment: Google Antigravity 2.0

System Overview and Problem Definition

LayoverNow is a Chrome extension that helps travelers turn an ordinary flight connection into a free, planned stopover. Several airlines, including Icelandair, TAP Portugal, Copa, and Emirates, offer free multi-day stopover programs, but most travelers never use them. The programs are hard to find, and there is no straightforward way to weigh the extra travel time of a detour against the value of the destination. LayoverNow addresses this by letting users enter an origin, destination, departure date, and desired stopover length, then surfacing routes whose layover cities are worth stopping in. The intended users are leisure travelers, budget-minded flyers, students, and anyone planning an international trip who wants to see more destinations for the same airfare.

Major Features

Airport search and offline database. The autocomplete search runs against a local geocoding database that covers every U.S. state and the major international hubs. Typing a two-letter state code such as "TX" filters to every Texas airport. Keeping the search fully offline makes it instant and removes a network dependency from the main workflow.

Detour and stopover scoring engine. The system uses the Haversine great-circle distance formula to estimate detour overhead, and scores each candidate layover hub against known airline stopover programs. Every route is rated on how far it bends the trip and how worthwhile the stop is.

Ranking modes and visualization. Results can be ranked by Best Value, Savings, or Minimum Detour. An interactive flight radar shows candidate routes, and multi-city booking links are generated for Skyscanner and Google Flights so the user can go from result to booking in one click.

AeroAI Advisor. An AI-assisted advisor evaluates each candidate layover for feasibility based on layover length, season, and traveler interests. It also pulls in trip context like visa status, transit currency, and weather forecast.

Versioning and Incremental Development

Version 1: Deterministic layover finder. V1 is a complete, self-contained tool. It includes airport search, the Haversine detour engine, static stopover-program scoring, the flight radar visualization, and booking link generation. With no network dependency, it can be tested in isolation. V1 will be built and put through a full system test before any V2 work begins.

Version 2: AI-assisted advisory layer. V2 introduces substantial new decision logic. The AeroAI Advisor calls an external AI API to produce a reasoned, personalized assessment of each layover, and live context enrichment adds visa, currency, and weather data. The system shifts from sorting routes by a fixed formula to producing recommendations a static table cannot. All network and AI dependencies are deliberately kept inside V2, so failures stay isolated from the proven V1 core. V2 will be built and fully system tested after V1 is stable.

The boundary between the two versions is intentional. V1 has to stand on its own as a working, deterministic product. V2 layers intelligence on top once the foundation is verified. The change between versions is a shift in decision logic and workflow, not a cosmetic update.

System Testing Awareness

Each version is built and then fully system tested before the next one starts. V1 testing is deterministic and repeatable. It verifies routing math, autocomplete behavior, ranking output, and link generation, all offline. V2 testing is much more involved. On top of regression-testing every V1 path, it has to cover API latency, rate limits, malformed or missing responses, and graceful fallback to V1's deterministic ranking when a service is unavailable.

Smart or Intelligent System Justification

V1 makes no claim of intelligence. It is deterministic and rules-driven by design. The "smart" label applies only to V2, and it is justified through AI-assisted decision support. The AeroAI Advisor hands off a real judgment, whether a given layover is worth leaving the airport for, to an external AI API. The output is a personalized recommendation rather than a fixed numeric score.

Engineering Practices: Risk and Version Control

Source code lives in a GitHub repository, and the instructor has access. Change control uses feature branches, one commit per logical change, and tagged releases (v1.0, v2.0) marking each system-tested version.

Tracked project risks include scope creep and Manifest V3 browser-API changes. The key version-specific risk sits in V2: a dependency on third-party AI and data APIs, with exposure to rate limits, API-key cost, latency, and outages. The mitigation is response caching and an automatic fallback to V1's deterministic ranking when an API is down.